# ASSIGNMENT-20.3

NAME :-P.CHARANKUMAR

ROLL NO :-2303A52138

BATCH :-41

## Task 1 – Password Strength & Validation Check

## PROMPT: Password Strength & Validation Check

# Requirements:

# 1. Create a function to check the strength of a password based on the following criteria

#  - Minimum length of 8 characters.

#  - Contains at least one uppercase letter.

#  - Contains at least one lowercase letter.

#  - Contains at least one digit.

# 2. The function should return a message indicating whether the password is "Strong", "Moderate", or "Weak".

# 3. Handle edge cases (e.g., empty string, only special characters) appropriately

E: > ai ac.py > ...

```python
#Password Strength & Validation Check
# Requirements:
# 1. Create a function to check the strength of a password based on the following criteria
#    - Minimum length of 8 characters.
#    - Contains at least one uppercase letter.
#    - Contains at least one lowercase letter.
#    - Contains at least one digit.
# 2. The function should return a message indicating whether the password is "Strong", "Moderate", or "Weak".
# 3. Handle edge cases (e.g., empty string, only special characters) appropriately
def check_password_strength(password):
    """
    Check the strength of a given password based on specific criteria.
    Parameters:
    password (str): The password string to be evaluated.
    Returns:
    str: A message indicating whether the password is "Strong", "Moderate", or "Weak".
    Criteria:
    - Strong: Meets all criteria (length >= 8, contains uppercase, lowercase, and digit).
    - Moderate: Meets at least two of the criteria.
    - Weak: Meets fewer than two criteria.
    """
    # Initialize criteria counters
    has_uppercase = any(char.isupper() for char in password)
    has_lowercase = any(char.islower() for char in password)
    has_digit = any(char.isdigit() for char in password)
    # Check the length of the password
    is_long_enough = len(password) >= 8
```

```python
    """
    # Initialize criteria counters
    has_uppercase = any(char.isupper() for char in password)
    has_lowercase = any(char.islower() for char in password)
    has_digit = any(char.isdigit() for char in password)
    # Check the length of the password
    is_long_enough = len(password) >= 8
    # Count how many criteria are met
    criteria_met = sum([has_uppercase, has_lowercase, has_digit, is_long_enough])
    # Determine password strength based on criteria met
    if criteria_met == 4:
        return "Strong"
    elif criteria_met >= 2:
        return "Moderate"
    else:
        return "Weak"
# Example usage:
if __name__ == "__main__":
    test_password = "yashwanth@2486"
    print(f"The password '{test_password}' is: {check_password_strength(test_password)}")
```

OUTPUT:

```
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yashw\AppData
y"
The password 'Password@2486' is: Strong
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code>
```

## Task 2 – Insecure File Upload Handling

PROMPT: # Requirements:

# 1. Create a function to handle file uploads securely.

# 2. Validate the file type (e.g., only allow .jpg, .png, .pdf).

# 3. Check the file size (e.g., limit to 5MB).

# 4. Sanitize the file name to prevent directory traversal attacks.

# 5. Store the file in a designated directory with a unique name to avoid overwriting

```python
#Insecure File Upload handling
# Requirements:
# 1. Create a function to handle file uploads securely.
# 2. Validate the file type (e.g., only allow .jpg, .png, .pdf).
# 3. Check the file size (e.g., limit to 5MB).
# 4. Sanitize the file name to prevent directory traversal attacks.
# 5. Store the file in a designated directory with a unique name to avoid overwriting
import os
import uuid
def secure_file_upload(file_name, file_content):
    """
    Handle file uploads securely by validating file type, size, and sanitizing the file name.
    Parameters:
    file_name (str): The original name of the uploaded file.
    file_content (bytes): The content of the uploaded file.
    Returns:
    str: A message indicating the result of the upload process.
    Criteria:
    - Allowed file types: .jpg, .png, .pdf
    - Maximum file size: 5MB
    - Sanitized file name to prevent directory traversal attacks.
    - Unique file name to avoid overwriting existing files.
    """
    # Define allowed file extensions and maximum file size
    allowed_extensions = {'.jpg', '.png', '.pdf'}
    max_file_size = 5 * 1024 * 1024  # 5MB in bytes
    # Extract the file extension
    _, file_extension = os.path.splitext(file_name)
    # Validate the file type
    if file_extension.lower() not in allowed_extensions:
        return "Invalid file type. Only .jpg, .png, and .pdf are allowed."
```

```python
    if len(file_content) > max_file_size:
        return "File size exceeds the limit of 5MB."
    # Sanitize the file name to prevent directory traversal attacks
    sanitized_file_name = os.path.basename(file_name)
    # Generate a unique file name to avoid overwriting existing files
    unique_file_name = f"{uuid.uuid4()}{file_extension}"
    # Define the directory to store uploaded files
    upload_directory = "uploads"
    # Ensure the upload directory exists
    os.makedirs(upload_directory, exist_ok=True)
    # Save the file securely
    with open(os.path.join(upload_directory, unique_file_name), 'wb') as f:
        f.write(file_content)
    return f"File uploaded successfully as {unique_file_name}."
# Example usage:
if __name__ == "__main__":
    # Simulate file upload
    test_file_name = "example.pdf"
    test_file_content = b"%PDF-1.4 example content"  # Simulated file content in bytes
    print(secure_file_upload(test_file_name, test_file_content))
```

OUTPUT:

```
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yashw\AppData\Local\Programs\Python\Python313\python.exe "e:/ai ac.p
y"
File uploaded successfully as 8f6d3cc4-34ce-460c-a721-87a0e9f82f01.pdf.
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code>
```

## Task 3 – Real-Time Application: API Security Review

# Requirements:

# 1. Create a function to review API security based on common vulnerabilities.

# 2. Check for vulnerabilities such as:

#  - SQL Injection

#  - Cross-Site Scripting (XSS)

#  - Insecure Direct Object References (IDOR)

# 3. The function should return a report indicating the presence of any vulnerabilities and recommendations for mitigation

```python

#Real-Time Application: API Security review
# Requirements:
# 1. Create a function to review API security based on common vulnerabilities.
# 2. Check for vulnerabilities such as:
#    - SQL Injection
#    - Cross-Site Scripting (XSS)
#    - Insecure Direct Object References (IDOR)
# 3. The function should return a report indicating the presence of any vulnerabilities and recommendations for mitigation.
def review_api_security(api_endpoint):
    """
    Review API security based on common vulnerabilities such as SQL Injection, XSS, and IDOR.
    Parameters:
    api_endpoint (str): The API endpoint to be reviewed for security vulnerabilities.
    Returns:
    str: A report indicating the presence of any vulnerabilities and recommendations for mitigation.
    """
    # Simulated vulnerability checks (In a real implementation, this would involve more complex logic)
    vulnerabilities = []
    # Check for SQL Injection vulnerability
    if "SELECT" in api_endpoint.upper() or "DROP" in api_endpoint.upper():
        vulnerabilities.append("SQL Injection detected. Recommendation: Use parameterized queries.")
    # Check for Cross-Site Scripting (XSS) vulnerability
    if "<script>" in api_endpoint.lower():
        vulnerabilities.append("XSS detected. Recommendation: Sanitize user input and use Content Security Policy (CSP).")
    # Check for Insecure Direct Object References (IDOR) vulnerability
    if "/user/" in api_endpoint.lower() and any(char.isdigit() for char in api_endpoint):
        vulnerabilities.append("IDOR detected. Recommendation: Implement proper access controls and authorization checks.")
    # Generate the report
    if vulnerabilities:
        report = "Vulnerabilities found:\n" + "\n".join(vulnerabilities)
    else:
        report = "No vulnerabilities detected. The API endpoint appears to be secure."
    return report
```

```python
# Example usage:
if __name__ == "__main__":
    test_api_endpoint = "/user/123?name=<script>alert('XSS')</script>"
    print(review_api_security(test_api_endpoint))
```

OUTPUT:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   powershell

PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yashw\AppData\Local\Programs\Python\Python313\python.exe "e:/ai ac.py"
Vulnerabilities found:
XSS detected. Recommendation: Sanitize user input and use Content Security Policy (CSP).
IDOR detected. Recommendation: Implement proper access controls and authorization checks.
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code>
```

Task 4 – Cross-Site Scripting (XSS) Check

PROMPT: Requirements:

# 1. Create a function to check for potential XSS vulnerabilities in user input.

# 2. The function should analyze the input for common XSS attack vectors (e.g., <script> tags, event handlers).

# 3. The function should return a message indicating whether the input is safe or potentially vulnerable to XSS attacks

```python
    print(review_api_security(test_api_endpoint))'''
#Cross-Site Scripting (XSS) Check
# Requirements:
# 1. Create a function to check for potential XSS vulnerabilities in user input.
# 2. The function should analyze the input for common XSS attack vectors (e.g., <script> tags, event handlers).
# 3. The function should return a message indicating whether the input is safe or potentially vulnerable to XSS attacks.
def check_xss_vulnerability(user_input):
    """
    Check for potential Cross-Site Scripting (XSS) vulnerabilities in user input.
    Parameters:
    user_input (str): The input string to be analyzed for XSS vulnerabilities.
    Returns:
    str: A message indicating whether the input is safe or potentially vulnerable to XSS attacks.
    """
    # Define common XSS attack vectors
    xss_attack_vectors = ["<script>", "onerror=", "onload=", "javascript:"]
    # Check if any of the attack vectors are present in the user input
    for vector in xss_attack_vectors:
        if vector in user_input.lower():
            return "Potential XSS vulnerability detected. Input is potentially unsafe."
    return "Input is safe from XSS vulnerabilities."
# Example usage:
if __name__ == "__main__":
    test_input = "<script>alert('XSS')</script>"
    print(check_xss_vulnerability(test_input))
```

OUTPUT:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS        powershell
                    Open file in editor (ctrl + click)

PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yashw\AppData\Local\Programs\Python\Python313\python.exe "e:/ai ac.py"
Potential XSS vulnerability detected. Input is potentially unsafe.
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code>
```

Task 5 – SQL Injection Prevention

PROMPT: Requirements:

# 1. Create a function to prevent SQL injection attacks by sanitizing user input.

# 2. The function should escape special characters and use parameterized queries

```python

# SQL Injection Prevention
# Requirements:
# 1. Create a function to prevent SQL injection attacks by sanitizing user input.
# 2. The function should escape special characters and use parameterized queries.
def prevent_sql_injection(user_input):
    """
    Prevent SQL injection attacks by sanitizing user input.
    Parameters:
    user_input (str): The input string to be sanitized for SQL injection prevention.
    Returns:
    str: A sanitized version of the input string that is safe for use in SQL queries.
    """
    # Escape special characters commonly used in SQL injection attacks
    sanitized_input = user_input.replace("'", "''").replace(";", "").replace("--", "")
    return sanitized_input
# Example usage:
if __name__ == "__main__":
    test_input = "O'Reilly; DROP TABLE users; --"
    print(prevent_sql_injection(test_input))
```

OUTPUT:

```
    print(prevent_sql_injection(test_input))
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS        powershell

PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yashw\AppData\Local\Programs\Python\Python313\python.exe "e:/ai ac.py"
O''Reilly DROP TABLE users
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code>
```